AGENT02: Python Implementation

Reinforcement-Learning-Based Agents and Rule-Based Agents

Intelligent System Development

Academic Year 2026–2027

Purpose. This document accompanies Laboratory Experiment 02. It describes the Python implementation of two decision-making approaches in a shared e-commerce environment.
Python source file: `agent02_experiment.py`
Dependencies: NumPy and Matplotlib.
Execution command: `python agent02_experiment.py`
Laboratory report deadline: 11:30 PM, Thursday, 29 October 2026.
Required report filename: `agent02_class_team_name.pdf`

Contents

1 Overview **3**

2 Software Requirements **3**

3 Mathematical Model **3**

3.1 State and action 3

3.2 Reward 4

3.3 Q-learning 4

4 Program Architecture **4**

5 Complete Python Source **5**

6 Experiments and Outputs **5**

6.1 Experiment 1: Baseline 5

6.2 Experiment 2: Learning rate 5

6.3 Experiment 3: Exploration 5

6.4 Experiment 4: Multiple random seeds 6

7 Output Interpretation **6**

7.1 Average reward 6

7.2 Action frequency 6

7.3 Q-table 6

8 Reproducibility and Limitations **7**

9 Student Tasks **7**

10 Conclusion **7**

1 Overview

The AGENT02 program implements and compares two agent types:

1. A reinforcement-learning-based agent using tabular Q-learning.
2. A rule-based agent using a predefined conditional policy.

Both agents interact with a simulated e-commerce environment. The RL agent learns action values from rewards, whereas the rule-based agent follows a fixed decision policy.

The program supports the following experiments:

- Baseline training and evaluation.
- Comparison of learning rates.
- Comparison of exploration schedules.
- Evaluation across multiple random seeds.
- Export of CSV files, a learned Q-table, and figures.

2 Software Requirements

Use Python 3.10 or a later compatible version.

Install the required packages:

```
pip install numpy matplotlib
```

Save the source file as `agent02_experiment.py`.

Execute the program:

```
python agent02_experiment.py
```

The program creates an output directory named `agent02_results`.

3 Mathematical Model

3.1 State and action

For agent i, the local state is

$$s_i(t) = (d_t, c_i(t)),$$

where $d_t \in \{0, 1\}$ is the demand level and $c_i(t) \in \{0, 1\}$ is the competitor's previous action.

The action space is

$$\mathcal{A} = \{0, 1\},$$

where action 0 means no discount and action 1 means offering a discount.

Each agent has four possible states and two possible actions. The Q-table therefore contains eight entries.

3.2 Reward

The simulated conversion probability is

$$p_i(t) = \text{clip}\left(0.30 + 0.25d_t + 0.15a_i(t) - 0.10a_{-i}(t),\, 0.05,\, 0.95\right).$$

The number of sales follows

$$S_i(t) \sim \text{Binomial}(N_t, p_i(t)),$$

where $N_t = 10$ for low demand and $N_t = 20$ for high demand.

With price $P = 10$, unit cost $C = 5$, and discount rate $\delta = 0.20$, the reward is

$$r_i(t) = S_i(t)\left[P(1 - \delta a_i(t)) - C\right].$$

This is a simplified teaching model. Its assumptions should not be interpreted as an empirically validated model of real e-commerce markets.

3.3 Q-learning

The RL agent updates its action-value estimate according to

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right].$$

Here, α is the learning rate and γ is the discount factor.

The RL policy uses an ε-greedy strategy during training. During the separate evaluation phase, the learned policy acts greedily and its Q-table is not updated.

4 Program Architecture

The source code contains the following components.

Component	Responsibility
`Config`	Stores simulation parameters and random seeds.
`ECommerceEnvironment`	Generates observations, sales, rewards, and next states.
`QLearningAgent`	Selects actions and updates the Q-table.
`RuleBasedAgent`	Selects actions using predefined rules.
`train_rl_against_rule`	Trains the RL agent against a fixed rule-based opponent.
`evaluate_pair`	Evaluates frozen policies without Q-table updates.
`run_baseline`	Runs the baseline experiment and saves results.
`run_learning_rate_experiment`	Compares learning-rate configurations.
`run_exploration_experiment`	Compares exploration schedules.
`run_multi_seed_experiment`	Measures variability across independent random seeds.

5 Complete Python Source

The complete executable source is maintained in the separate file `agent02_experiment.py`.

To include that source directly in this PDF, keep the Python file in the same directory as this LaTeX file and use:

```
\lstinputlisting[
    style=pythonstyle,
    caption={Complete AGENT02 Python implementation},
    label={lst:agent02}
]{agent02_experiment.py}
```

The command above includes the actual contents of the Python file at compilation time. It avoids maintaining two manually copied versions of the source.

If the Python file is not available in the same directory, the LaTeX compilation will fail at this point. Do not replace the missing source with invented or incomplete code.

6 Experiments and Outputs

6.1 Experiment 1: Baseline

The program trains the RL agent against the fixed rule-based agent and evaluates both policies in a separate run.

The evaluation disables exploration and does not update the learned Q-table.

The principal outputs are:

- `baseline_episode_rewards.csv`
- `baseline_evaluation.csv`
- `baseline_evaluation_episodes.csv`
- `q_tables.txt`
- `episode_rewards.png`

6.2 Experiment 2: Learning rate

The learning rates are

$$\alpha \in \{0.01, 0.10, 0.50\}.$$

The program records evaluation rewards for the resulting policies. Students should discuss the effects of the learning rate on updating and stability.

Output files include `learning_rate_comparison.csv` and `learning_rate_comparison.png`.

6.3 Experiment 3: Exploration

The program compares three exploration schedules:

Setting	Initial ε	Minimum	Decay
Low	0.05	0.02	0.999
Baseline	0.20	0.02	0.995
High	0.50	0.05	0.995

The resulting policies are evaluated without exploration. This separates the quality of the learned policy from random exploratory actions during evaluation.

6.4 Experiment 4: Multiple random seeds

The program repeats the baseline experiment using several training seeds. It reports the mean and sample standard deviation of evaluation rewards.

A single random seed may produce a misleading comparison because the environment includes stochastic demand and sales.

7 Output Interpretation

7.1 Average reward

For T evaluation steps, the average reward is

$$\overline{R}_i = \frac{1}{T} \sum_{t=1}^{T} r_i(t).$$

A larger value indicates greater simulated profit per time step under the specified environment and evaluation protocol.

It does not establish that the agent will perform better in a real marketplace or under a different environment.

7.2 Action frequency

The discount action rate is

$$\rho_i = \frac{\text{number of discount actions}}{\text{total number of actions}}.$$

A larger discount rate does not necessarily imply greater profit. The discount increases the conversion probability in the model but reduces the profit margin per sale.

7.3 Q-table

The Q-table stores the learned action values for each state. The greedy policy chooses the action with the largest estimated value.

Students should inspect the table and explain how its entries determine the learned policy.

8 Reproducibility and Limitations

The program specifies random seeds and records parameter configurations. Reproducibility also depends on the software environment and the consistency of the evaluation protocol.

The following limitations must be considered:

1. The market has only two demand states and two actions.
2. The conversion-probability equation is illustrative.
3. Demand evolves independently of the agents' actions.
4. The RL agent is trained against a fixed rule-based opponent rather than another simultaneously learning agent.
5. A tabular method does not scale directly to large, continuous state spaces.
6. The model does not include inventory, customer heterogeneity, operating costs beyond unit cost, or long-term customer relationships.

These limitations are appropriate for an introductory laboratory but should be addressed before using the model for applied economic conclusions.

9 Student Tasks

After running the program, students should:

1. Verify that all expected output files are generated.
2. Inspect the learned Q-table.
3. Compare baseline evaluation rewards.
4. Examine the learning-rate experiment.
5. Examine the exploration experiment.
6. Compare performance across random seeds.
7. Answer all ten questions in the AGENT02 assignment.
8. Explain at least one limitation and propose an improvement to the environment or agent.

Students must report actual measured results and should not invent numerical results or assume that the RL agent must outperform the rule-based agent.

10 Conclusion

This implementation provides a reproducible starting point for studying the difference between learning-based and rule-based decision-making in a multi-agent environment.

The RL agent estimates action values from reward feedback. The rule-based agent follows a predefined policy. The experimental framework supports comparisons of rewards, action frequencies, learning rates, exploration schedules, and variability across random seeds.

The scientific objective is to interpret the results carefully, identify the assumptions that influence them, and determine which conclusions are justified by the experiments.